

-  Global Multicloud Healthcare & Financial Transaction Clearing Engine
 -  Table of Contents
 -  Architecture Overview
 -  Application Architecture
 - What Runs Inside the Clusters
 - How Applications Are Built and Stored
 - Container Registries (Private Access Only)
 - Helm Chart Structure
 -  High-Level Topology
 -  Component Matrix
 -  Network Architecture
 - Cross-Cloud VPN Mesh
 - Subnet Architecture
 -  Data Flow Architecture
 -  Compliance Architecture
 - HIPAA Controls
 - SOX Controls
 - GDPR Controls
 -  Identity Architecture (Azure AD)
 -  Directory Structure
 -  Prerequisites
 - Required GitHub Secrets
 -  Deployment Guide
 - Phase 1: Bootstrap State Backends (One-time)
 - Phase 2: Provision Infrastructure
 - Phase 3: Build and Deploy Applications
 - Option A: Via CI/CD (Recommended)
 - Option B: Manual Local Deploy
 - Phase 4: Configure ArgoCD & GitOps
 -  GitOps & ArgoCD
 -  Disaster Recovery
 -  CI/CD Pipeline Summary

Global Multicloud Healthcare & Financial Transaction Clearing Engine

Production-grade multicloud infrastructure for ingesting, validating, and clearing sensitive medical billing transactions across AWS, Azure, and GCP.

 Terraform >=1.7.0

 AWS Primary Active

 Azure Hot Standby

 GCP Compliance

HIPAA | SOX | GDPR | PCI-DSS Compliant

Table of Contents

- [Architecture Overview](#)
- [Application Architecture](#)
- [High-Level Topology](#)
- [Component Matrix](#)
- [Network Architecture](#)
- [Data Flow Architecture](#)
- [Compliance Architecture](#)
- [Identity Architecture \(Azure AD\)](#)
- [Directory Structure](#)
- [Prerequisites](#)
- [Deployment Guide](#)
- [GitOps & ArgoCD](#)
- [Disaster Recovery](#)

Architecture Overview

This system implements a **three-cloud active/standby/compliance** architecture:

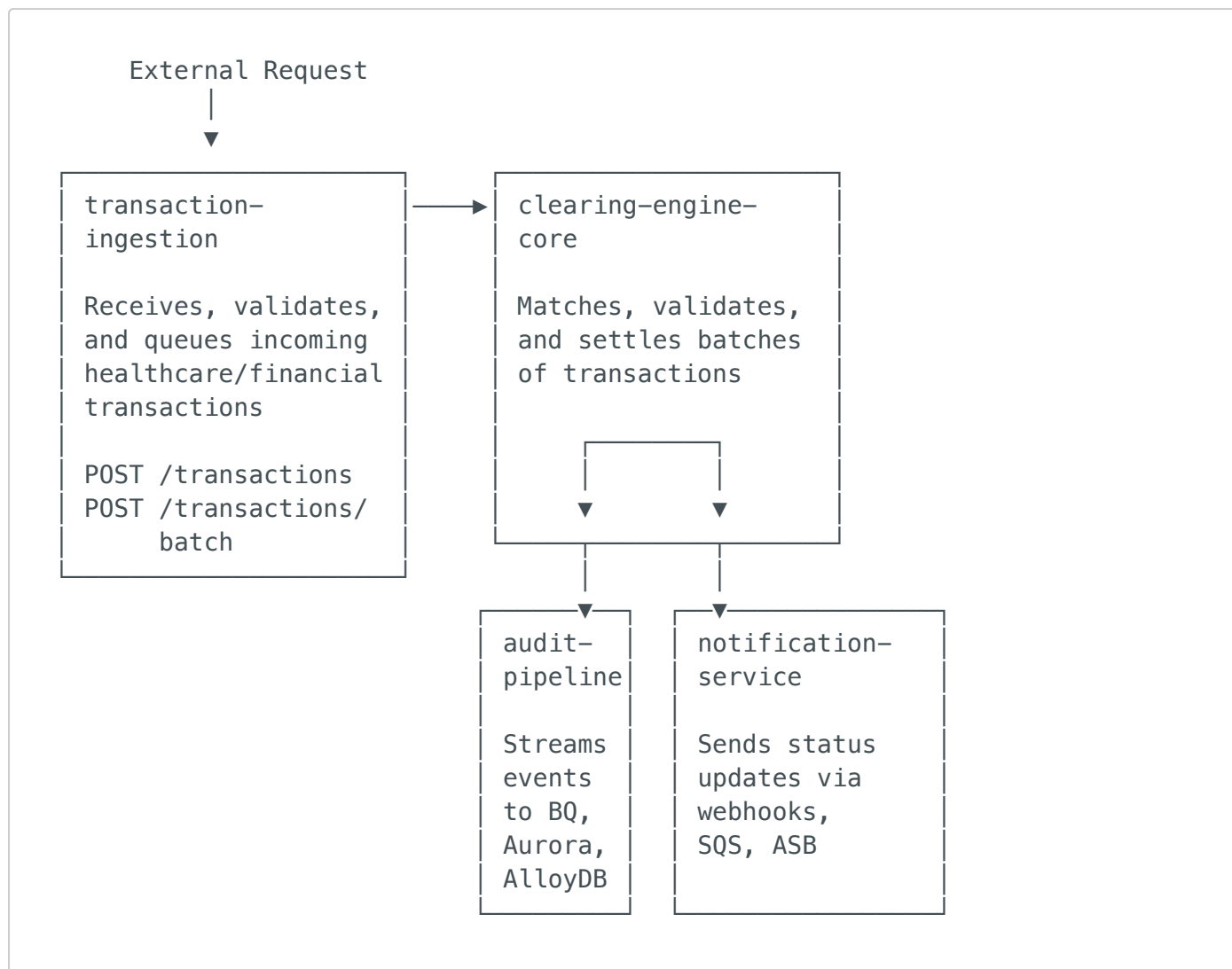
Cloud	Role	Services
AWS	Primary Active Site	EKS, Aurora PostgreSQL, ElastiCache Redis, CloudFront + WAF, Route 53, ECR
Azure	Hot Standby Site	AKS, Azure SQL Hyperscale, Azure Cache for Redis, Front Door + WAF, ACR
GCP	Compliance & Analytics	GKE, AlloyDB, BigQuery, Cloud Armor, Cloud KMS



Application Architecture

What Runs Inside the Clusters

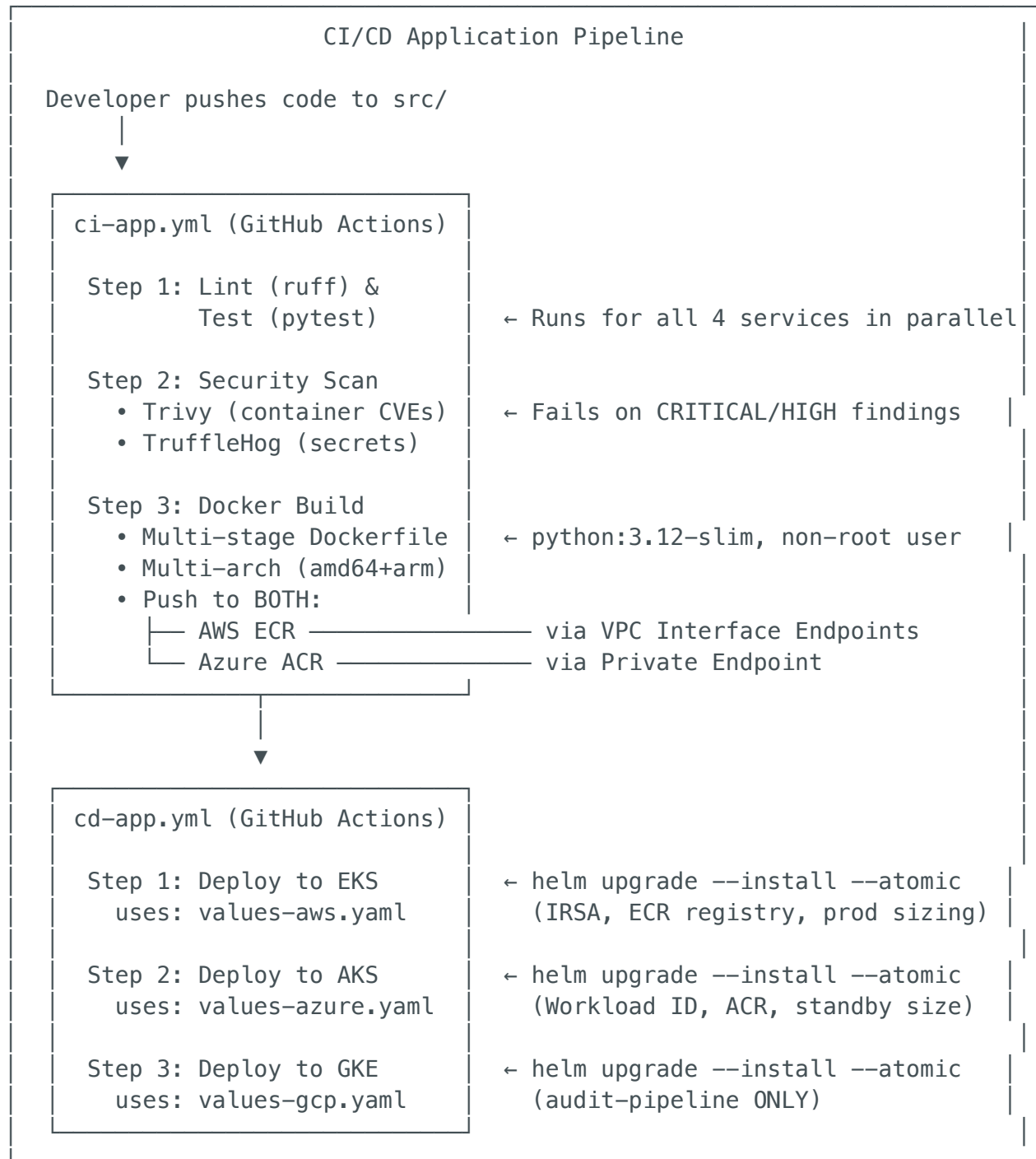
The clearing engine is composed of **4 Python/FastAPI microservices** that run as containers inside EKS, AKS, and GKE:



Service	Port	What It Does	Runs On
transaction-ingestion	8000	Receives transactions via REST, validates schemas, deduplicates via Redis, persists to DB	EKS + AKS
clearing-engine-core	8000	Core clearing logic — matches counterparties, validates business rules, settles batches	EKS + AKS

Service	Port	What It Does	Runs On
audit-pipeline	8000	Streams every transaction event to BigQuery (audit trail), AlloyDB (compliance), with PII tokenization (GDPR)	EKS + AKS + GKE
notification-service	8000	Sends transaction status updates via webhooks, SQS (AWS), Azure Service Bus, email	EKS + AKS

How Applications Are Built and Stored



Container Registries (Private Access Only)

Both EKS and AKS are **private clusters** with no Internet Gateway or NAT. Image pulls happen entirely within the private network:

Registry	Cloud	Access Method	Key Features
ECR (Elastic Container Registry)	AWS	VPC Interface Endpoints (ecr.api , ecr.dkr , s3)	KMS encryption, scan-on-push,

Registry	Cloud	Access Method	Key Features
			immutable tags, 30-image retention
ACR (Azure Container Registry)	Azure	Private Endpoint + Private DNS Zone (privatelink.azurecr.io)	Premium SKU, content trust, quarantine policy, AcrPull via managed identity

Helm Chart Structure

Applications are deployed using a **Helm umbrella chart** with 4 subcharts:

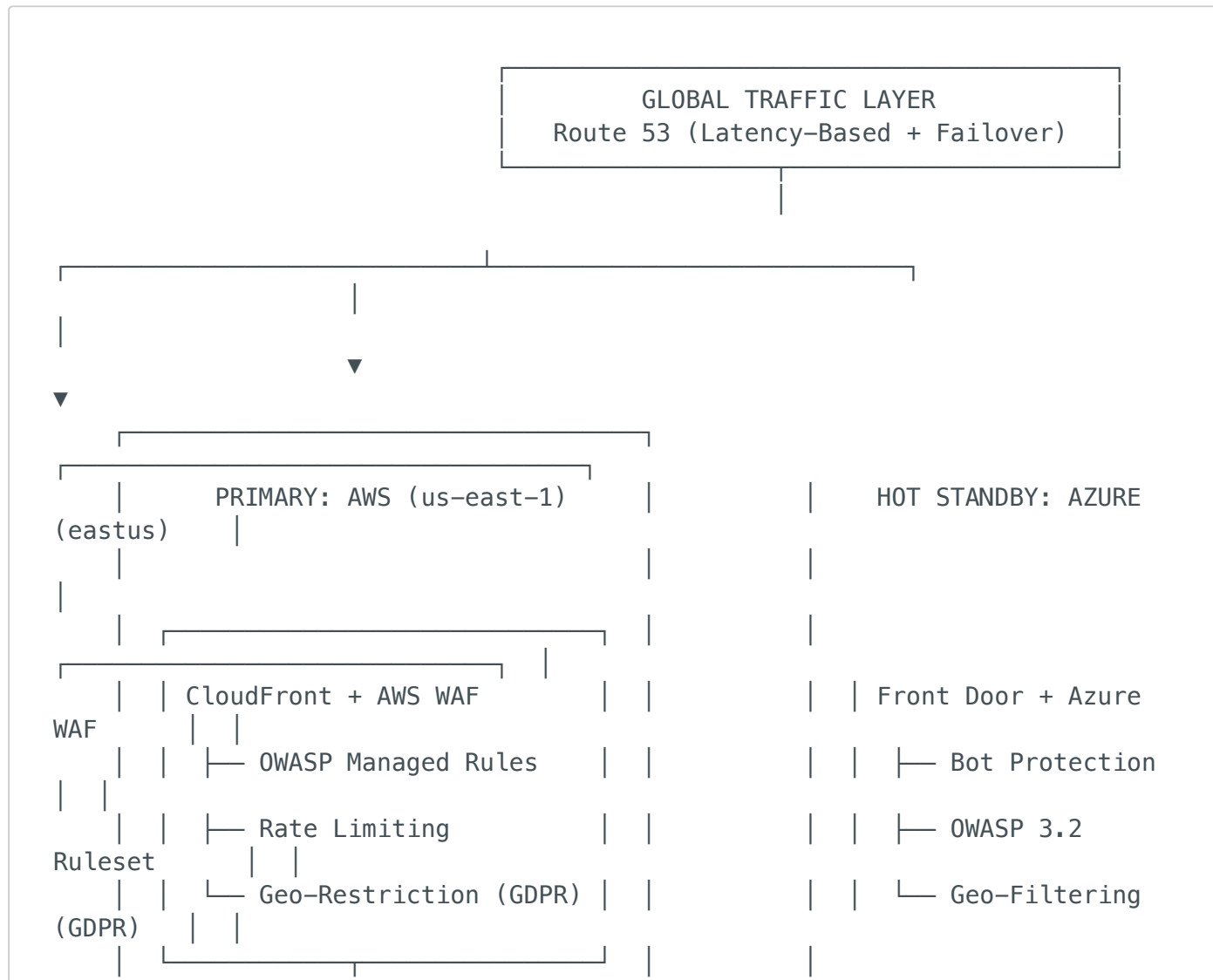
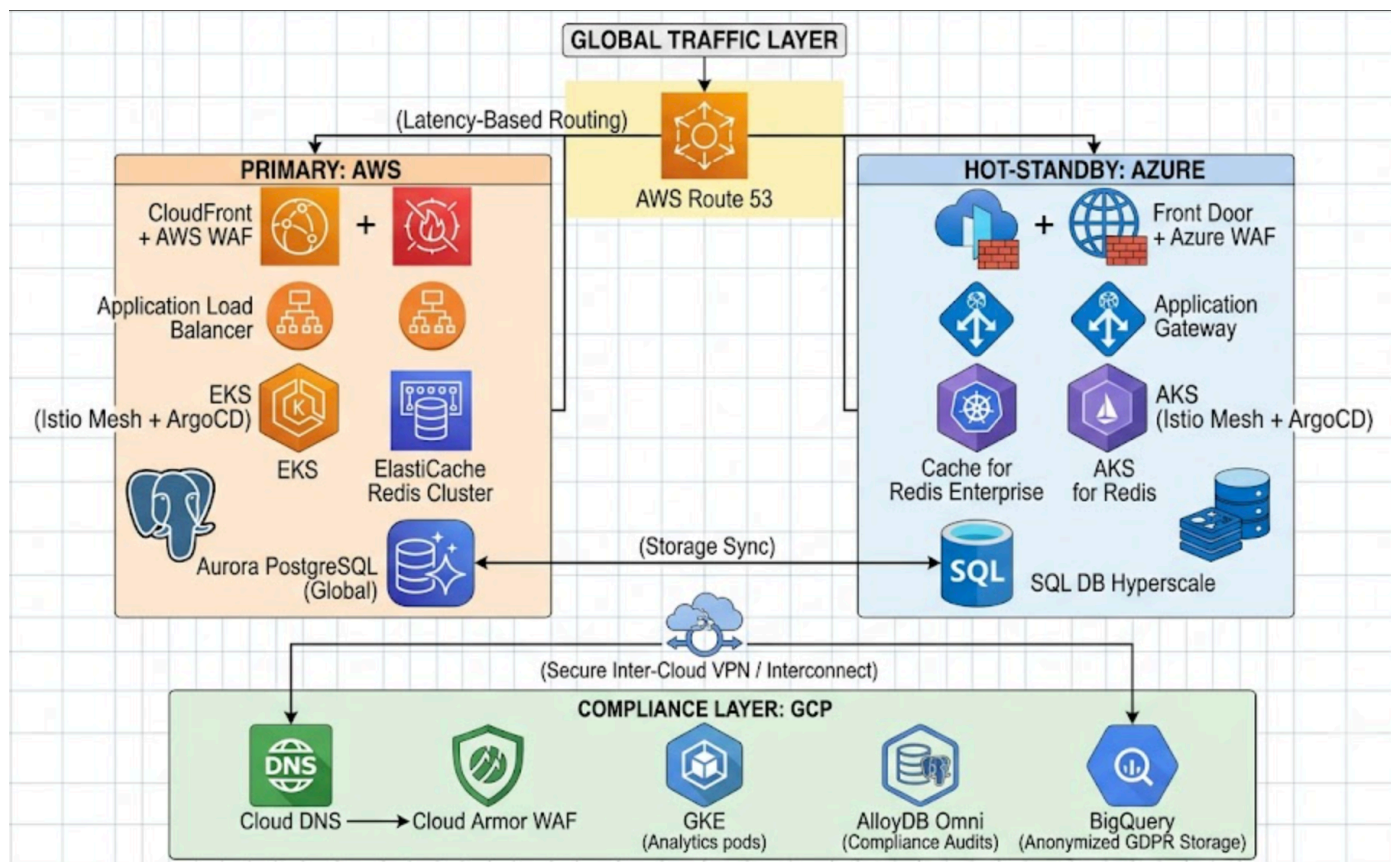
```
helm/clearing-engine/  
├─ Chart.yaml           # Umbrella chart (4 subchart dependencies)  
├─ values.yaml          # Base: replicas, resources, HPA, PDB  
├─ values-aws.yaml      # EKS: ECR images, IRSA annotations, prod  
sizing  
├─ values-azure.yaml    # AKS: ACR images, Workload Identity, standby  
sizing  
├─ values-gcp.yaml      # GKE: audit-pipeline ONLY, GCP Workload  
Identity  
├─ templates/  
├─┬─ network-policies.yaml # Zero-trust pod-to-pod rules  
├─┬─ charts/  
├─┬─ transaction-ingestion/ # Deployment, Service, HPA, PDB,  
ServiceAccount  
├─┬─ clearing-engine-core/  
├─┬─ audit-pipeline/  
├─┬─ notification-service/
```

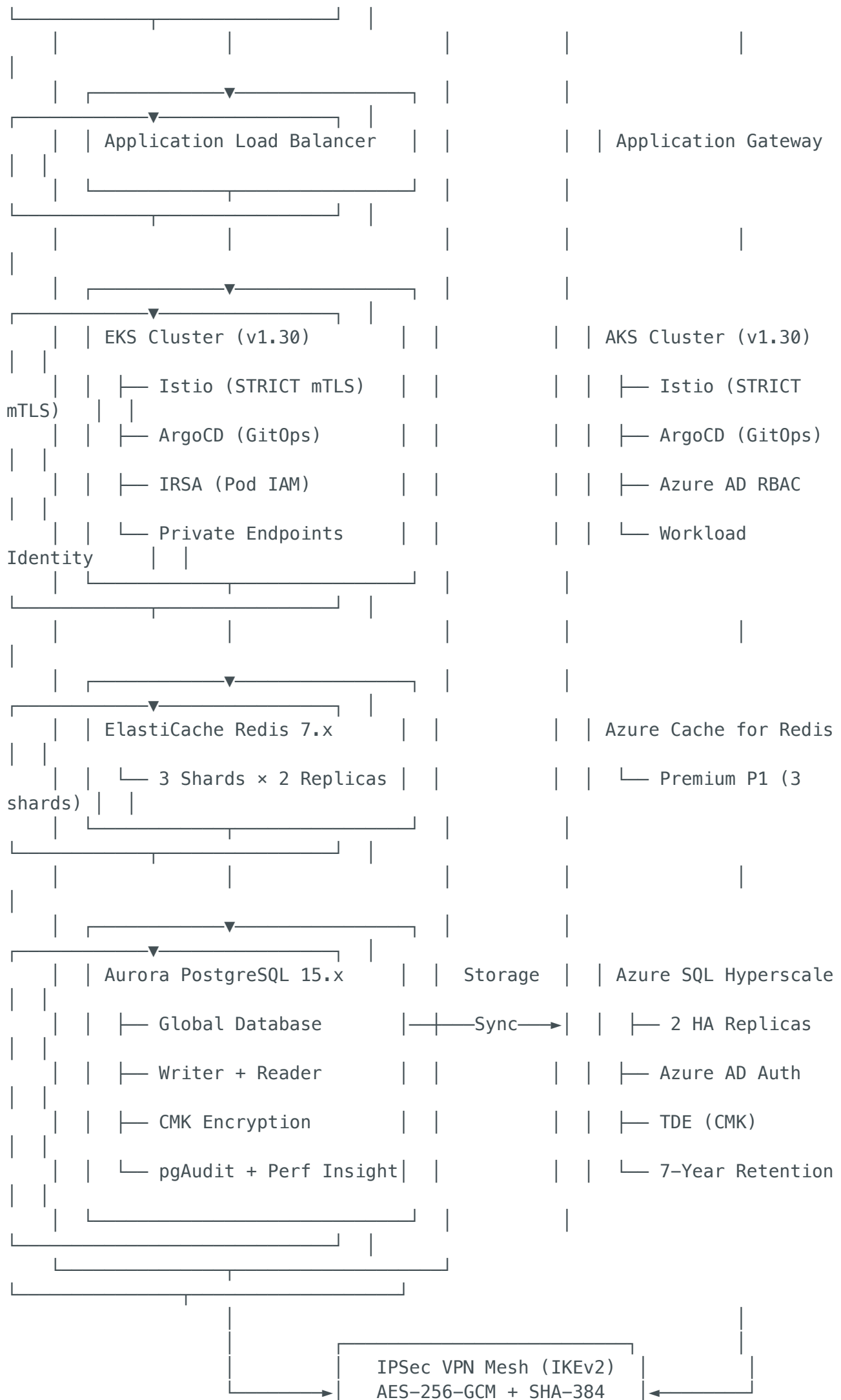
Each subchart Deployment includes:

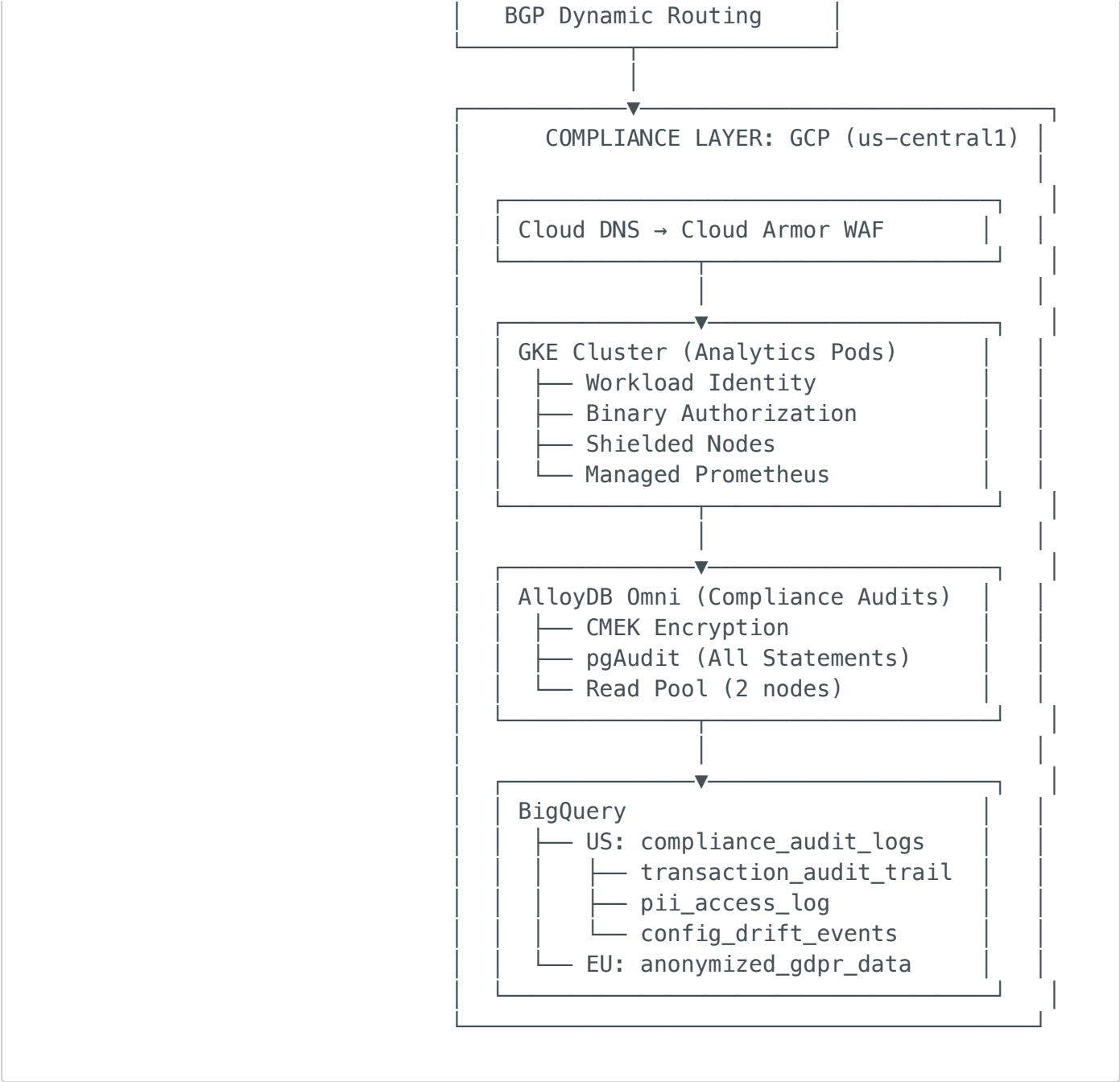
- **Security Context:** `runAsNonRoot`, `readOnlyRootFilesystem`, `allowPrivilegeEscalation: false`
- **Probes:** Liveness (`/health`), Readiness (`/ready`)
- **HPA:** Auto-scale at 70% CPU (3–15 replicas for ingestion, 3–20 for clearing)
- **PDB:** `minAvailable: 2` for zero-downtime upgrades
- **NetworkPolicies:** Only allowed paths: `ingestion` → `clearing` → `audit/notification`



High-Level Topology







Component Matrix

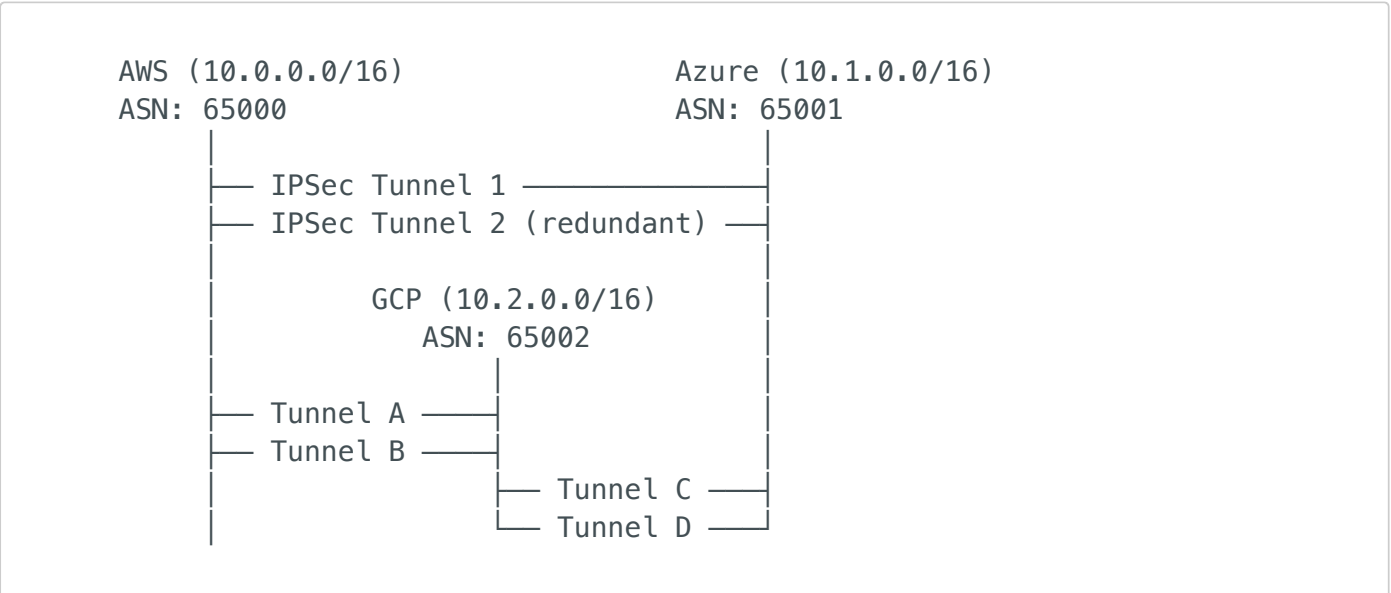
Component	AWS (Primary)	Azure (Hot Standby)	GCP (Compliance)
Global DNS	Route 53	Azure DNS	Cloud DNS
CDN & Edge	CloudFront + WAF	Front Door Premium + WAF	Cloud CDN + Armor
Compute	EKS + Managed Nodes	AKS + System/User Pools	GKE Standard

Component	AWS (Primary)	Azure (Hot Standby)	GCP (Compliance)
Service Mesh	Istio (STRICT mTLS)	Istio (STRICT mTLS)	Istio (STRICT mTLS)
GitOps	ArgoCD	ArgoCD	ArgoCD
Database	Aurora PostgreSQL 15	SQL Hyperscale	AlloyDB PostgreSQL
Caching	ElastiCache Redis 7.x	Cache for Redis Premium	—
Analytics	—	—	BigQuery
Secrets	KMS + IRSA	Key Vault + Workload ID	Cloud KMS + WI
VPN	VPN Gateway (BGP 65000)	VPN Gateway (BGP 65001)	HA VPN (BGP 65002)
Compliance	CloudTrail + Config	Azure Policy + Defender	Audit Logs + SCC
Identity	IAM + OIDC	Azure AD (Central)	IAM + WI Pools



Network Architecture

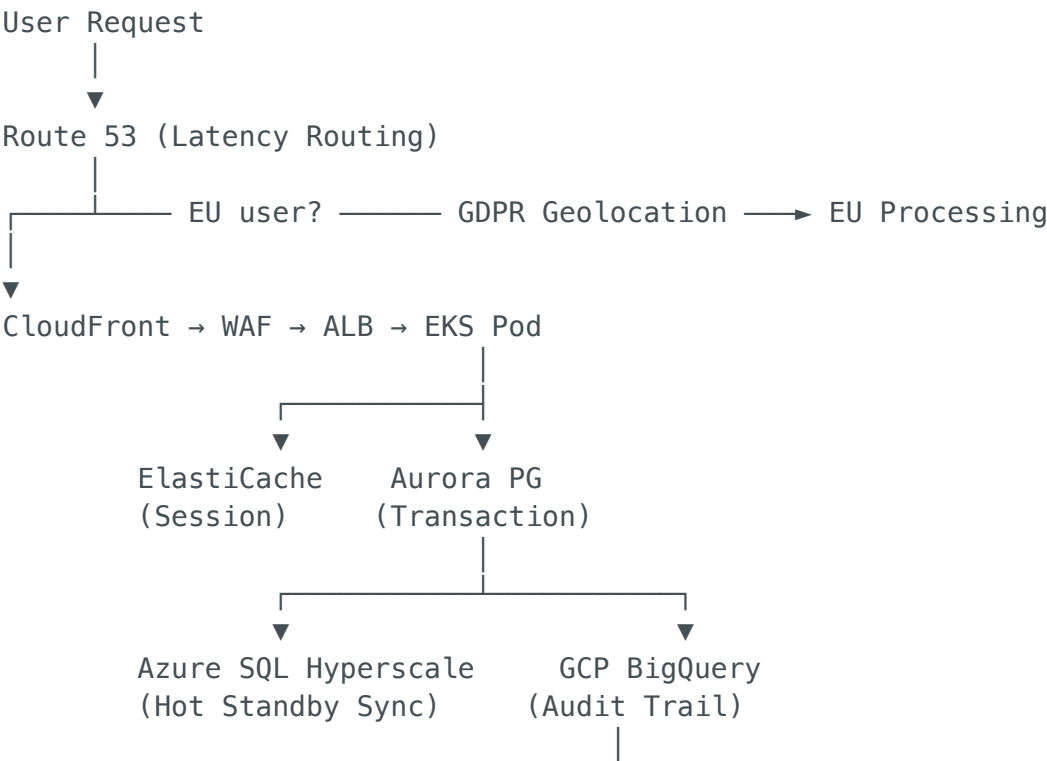
Cross-Cloud VPN Mesh



Subnet Architecture

Cloud	Subnet	CIDR	Purpose
AWS	Public (x2)	10.0.0.0/24, 10.0.1.0/24	ALB, NAT Gateways
AWS	Private-App (x2)	10.0.10.0/24, 10.0.11.0/24	EKS Worker Nodes
AWS	Private-Data (x2)	10.0.20.0/24, 10.0.21.0/24	Aurora, ElastiCache
Azure	AKS System	10.1.0.0/24	AKS System Pool
Azure	AKS User	10.1.1.0/24	AKS User Pool
Azure	Data	10.1.10.0/24	SQL, Redis, Key Vault
Azure	Gateway	10.1.255.0/24	VPN Gateway
GCP	GKE Nodes	10.2.0.0/20	GKE Node Pool
GCP	Data	10.2.16.0/20	AlloyDB, Private Services

Data Flow Architecture





Compliance Architecture

HIPAA Controls

Control	Implementation
Data at Rest Encryption	AWS KMS (CMK), Azure Key Vault (CMK), GCP Cloud KMS (CMEK)
Data in Transit	Istio mTLS (STRICT mode) + IPsec VPN (AES-256-GCM)
Access Controls	IAM Roles, Azure RBAC, GCP IAM + Workload Identity
Audit Logging	CloudTrail, Azure Monitor, GCP Audit Logs → BigQuery
Network Isolation	VPC/VNet with NACLs/NSGs, private subnets, no public DB access
Backup & Recovery	Aurora: 35-day, Azure SQL: 7-year, AlloyDB: 35-day + PITR

SOX Controls

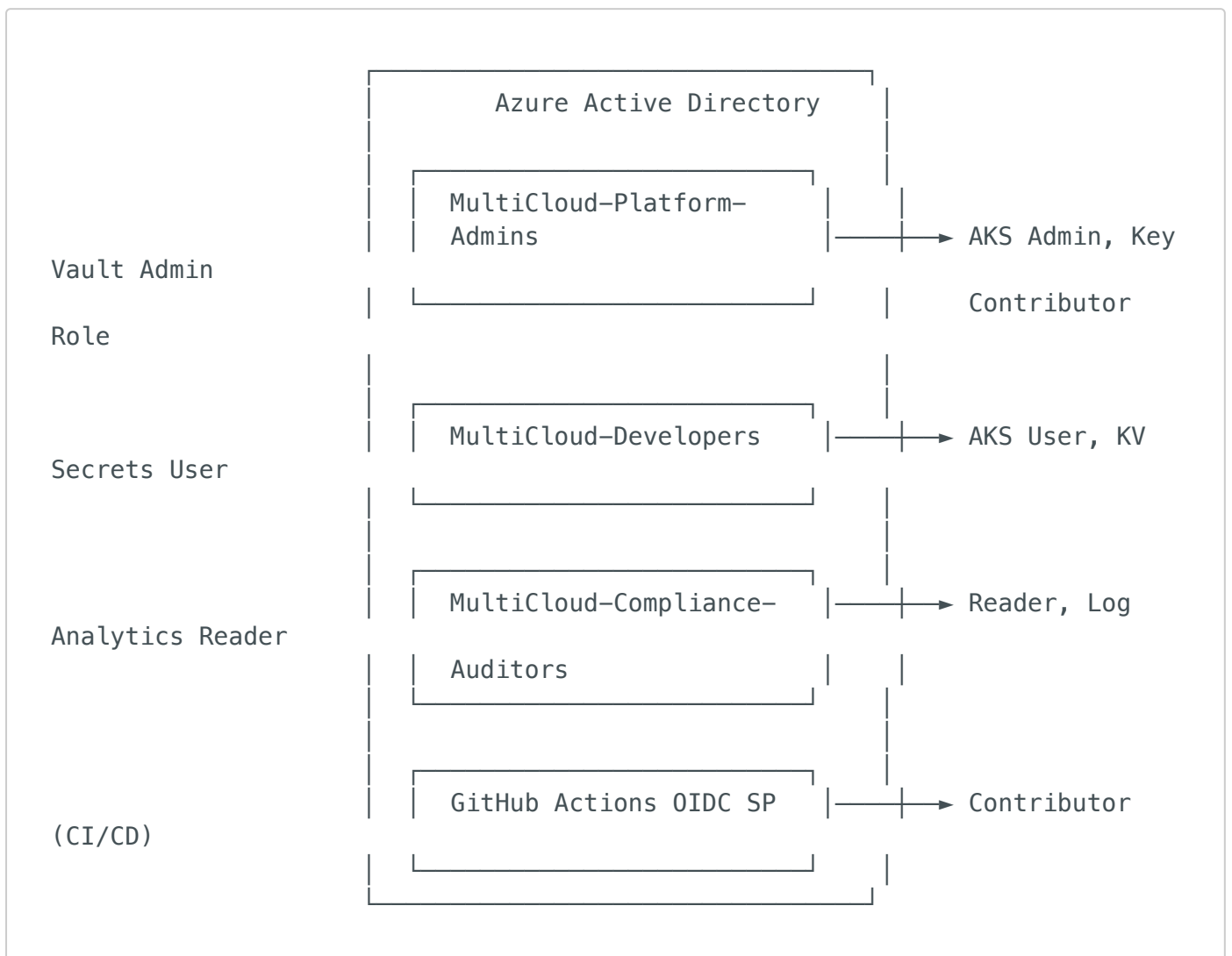
Control	Implementation
Immutable Audit Trail	CloudTrail (S3 + Glacier, 7yr), Azure Activity Log, GCP → BigQuery
Separation of Duties	Admin/Dev/Compliance Azure AD groups, IAM role boundaries
Change Detection	AWS Config Rules, Azure Policy, GCP Org Policies + Drift Alerts
Financial Auditability	pgAudit on all DB writes, Performance Insights, BigQuery analytics

GDPR Controls

Control	Implementation
Data Sovereignty	Route 53 geolocation routing, BigQuery EU dataset, KMS EU keyring
Right to Erasure	Tokenized PII, anonymization in microservices before cross-cloud sync
Egress Control	Istio Sidecar REGISTRY_ONLY mode + whitelisted ServiceEntries
Data Minimization	Pod-level egress blocks unauthorized API calls

Identity Architecture (Azure AD)

Azure Active Directory serves as the **centralized identity provider** across all three clouds:





Directory Structure

```
MultCloudProject/
├── .github/workflows/
│   ├── aws-deploy.yml           # Infra: AWS DevSecOps pipeline (Terraform)
│   └── azure-deploy.yml         # Infra: Azure DevSecOps pipeline
├── (Terraform)
│   ├── gcp-deploy.yml           # Infra: GCP DevSecOps pipeline (Terraform)
│   ├── ci-app.yml               # App: Build, test, scan, push to ECR+ACR
│   └── cd-app.yml               # App: Deploy Helm chart to EKS→AKS→GKE
├── src/                          # Application microservices
├── (Python/FastAPI)
│   ├── transaction-ingestion/    # Receives & validates transactions
│   │   └── Dockerfile           # Multi-stage, non-root, python:3.12-
│   │                               slim
│   │   ├── requirements.txt      # FastAPI, asyncpg, redis, structlog
│   │   └── app/
│   │       └── main.py           # FastAPI entry point + Prometheus
│   ├── metrics
│   │   └── config.py            # Pydantic Settings (env-driven, no
│   │                               secrets)
│   ├── routes/
│   │   └── health.py            # /health (liveness), /ready
│   ├── (readiness)
│   │   ├── transactions.py      # POST /api/v1/transactions
│   │   ├── models/              # Pydantic request/response schemas
│   │   ├── services/            # DB, Redis, validation logic
│   │   └── tests/                # pytest unit tests
│   ├── clearing-engine/          # Core clearing & settlement logic
│   ├── audit-pipeline/           # Compliance event streaming (→ BigQuery)
│   ├── notification-service/     # Webhooks, SQS, Azure Service Bus
│   ├── helm/                    # Kubernetes deployment (Helm v3)
│   │   ├── clearing-engine/      # Umbrella chart
│   │   │   ├── Chart.yaml        # 4 subchart dependencies
│   │   │   ├── values.yaml       # Base: resources, HPA, PDB
│   │   │   ├── values-aws.yaml   # EKS: ECR images, IRSA annotations
│   │   │   ├── values-azure.yaml # AKS: ACR images, Workload Identity
│   │   │   ├── values-gcp.yaml   # GKE: audit-pipeline only
│   │   │   └── templates/
│   │   │       ├── network-policies.yaml # Zero-trust pod-to-pod rules
│   │   │       └── charts/         # Per-service subcharts
│   │   │           ├── transaction-ingestion/ # Deployment, Service, HPA, PDB, SA
│   │   │           ├── clearing-engine-core/
│   │   │           ├── audit-pipeline/
│   │   │           └── notification-service/
│   ├── terraform/               # Infrastructure as Code
│   │   ├── bootstrap/           # State backend provisioning (run once)
│   │   │   ├── main.tf           # S3+DynamoDB, Azure Storage, GCS
│   │   │   ├── variables.tf
│   │   │   ├── outputs.tf
│   │   │   └── providers.tf
│   │   └── modules/
│   │       └── aws_infra/        # 15 files: VPC, EKS, Aurora, ECR, VPC
└── Endpoints...
```

```

|   |   |— azure_infra/           # 13 files: VNet, AKS, SQL, ACR, Private
Endpoints...
|   |   |— gcp_infra/           # 10 files: VPC, GKE, AlloyDB,
BigQuery...
|   |   |— environments/
|   |       |— production/       # Root module: wires all 3 cloud modules
|— gitops/                       # ArgoCD + Istio service mesh config
|   |— apps/
|       |— base/                 # Namespace + Istio STRICT mTLS
|       |— aws/                  # ArgoCD app + Istio egress for EKS
|       |— azure/                # ArgoCD app + Istio egress for AKS
|       |— gcp/                  # ArgoCD app + Istio egress for GKE
|— docs/
|   |— RUNBOOK.md               # Operations Runbook (650+ lines)
|— README.md

```

✓ Prerequisites

1. **Cloud Accounts:** Active AWS, Azure, and GCP accounts with billing enabled
2. **Terraform:** $\geq 1.7.0$ installed locally
3. **Helm:** $\geq 3.15.0$ installed locally
4. **Python:** ≥ 3.12 (for local development/testing)
5. **Docker:** For building container images locally
6. **OIDC Federation** (for CI/CD):
 - AWS: IAM OIDC Identity Provider + IAM Role for GitHub Actions
 - Azure: App Registration + Federated Credentials
 - GCP: Workload Identity Pool + Provider
7. **DNS Domain:** A registered domain for Route 53 hosted zone
8. **GitHub Secrets:** Configure all required secrets (see below)

Required GitHub Secrets

Secret	Purpose
<code>AWS_ROLE_ARN</code>	OIDC role for AWS authentication
<code>AZURE_CLIENT_ID</code> / <code>AZURE_TENANT_ID</code> / <code>AZURE_SUBSCRIPTION_ID</code>	Azure OIDC

Secret	Purpose
GCP_WORKLOAD_IDENTITY_PROVIDER / GCP_SERVICE_ACCOUNT	GCP OIDC
ACR_NAME / ACR_LOGIN_SERVER	Azure Container Registry
EKS_CLUSTER_NAME / AKS_CLUSTER_NAME / GKE_CLUSTER_NAME	Cluster names
AURORA_WRITER_ENDPOINT / AZURE_SQL_ENDPOINT / ALLOYDB_PRIVATE_IP	DB endpoints
ELASTICACHE_ENDPOINT / AZURE_REDIS_ENDPOINT	Cache endpoints
VPN_PSK_AWS_AZURE / VPN_PSK_AWS_GCP / VPN_PSK_AZURE_GCP	VPN pre-shared keys

Deployment Guide

Deployment happens in 3 phases: **Bootstrap** → **Infrastructure** → **Application**.

Phase 1: Bootstrap State Backends (One-time)

```
cd terraform/bootstrap
terraform init
terraform apply
# Note the output – copy S3/DynamoDB names into
environments/production/providers.tf
```

Phase 2: Provision Infrastructure

```
cd terraform/environments/production
vim terraform.tfvars          # Set your actual values
terraform init
```



```
terraform plan -out=tfplan
terraform apply tfplan
```

This provisions across all 3 clouds:

- VPCs/VNets, subnets, security groups
- EKS, AKS, GKE clusters (private, no public endpoints)
- Aurora, Azure SQL, AlloyDB databases
- ElastiCache, Azure Redis
- ECR (+ 9 VPC endpoints), ACR (+ Private Endpoint)
- Cross-cloud VPN mesh (6 tunnels)

Phase 3: Build and Deploy Applications

Option A: Via CI/CD (Recommended)

Push code to **main** — the pipelines handle everything:

```
src/ change → ci-app.yml runs:
├─ Lint + Test (all 4 services in parallel)
├─ Trivy container scan
└─ Build + Push to ECR and ACR

ci-app.yml success → cd-app.yml runs:
├─ helm upgrade → EKS (primary, all services)
├─ helm upgrade → AKS (standby, all services)
└─ helm upgrade → GKE (audit-pipeline only)
```

Option B: Manual Local Deploy

```
# 1. Build images locally
for svc in transaction-ingestion clearing-engine audit-pipeline
notification-service; do
  docker build -t $svc:latest src/$svc/
done

# 2. Tag and push to ECR
aws ecr get-login-password | docker login --username AWS --password-stdin
<ECR_REGISTRY>
for svc in transaction-ingestion clearing-engine audit-pipeline
notification-service; do
  docker tag $svc:latest <ECR_REGISTRY>/multicloud-clearing-
engine/$svc:latest
  docker push <ECR_REGISTRY>/multicloud-clearing-engine/$svc:latest
```

done

```
# 3. Deploy to EKS via Helm
aws eks update-kubeconfig --name <CLUSTER_NAME>
helm upgrade --install clearing-engine helm/clearing-engine/ \
  --namespace clearing-engine --create-namespace \
  -f helm/clearing-engine/values.yaml \
  -f helm/clearing-engine/values-aws.yaml \
  --set global.imageRegistry=<ECR_REGISTRY> \
  --wait --atomic

# 4. Verify
kubectl get pods -n clearing-engine
kubectl logs deploy/transaction-ingestion -n clearing-engine
```

Phase 4: Configure ArgoCD & GitOps

```
# On each cluster:
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-
cd/stable/manifests/install.yaml

# Apply mesh and GitOps config
kubectl apply -f gitops/apps/base/      # Namespace + Istio mTLS
kubectl apply -f gitops/apps/<cloud>/    # ArgoCD app + egress rules
```



GitOps & ArgoCD

ArgoCD manages the Kubernetes state after Terraform provisions infrastructure:

- **Auto-sync:** Changes pushed to **gitops/apps/** are automatically applied
- **Self-heal:** Manual cluster changes are reverted to Git state
- **Prune:** Orphaned resources are automatically cleaned up



Disaster Recovery

Scenario	RTO	RPO	Procedure
AWS Region Failure	5 min	0 (sync replication)	Route 53 auto-failover to Azure
Single AZ Failure	0	0	Multi-AZ architecture handles automatically
Database Corruption	30 min	5 min (PITR)	Point-in-time recovery from Aurora
Full Cloud Outage	15 min	~1 min	Manual DNS cutover to Azure Front Door



CI/CD Pipeline Summary

Pipeline	File	Trigger	What It Does
AWS Infra	<code>aws-deploy.yml</code>	<code>terraform/**</code> changes	Checkov → Plan → Apply → ArgoCD sync
Azure Infra	<code>azure-deploy.yml</code>	<code>terraform/modules/azure_infra/**</code>	Checkov+tfsec → Plan → Apply → Compliance report
GCP Infra	<code>gcp-deploy.yml</code>	<code>terraform/modules/gcp_infra/**</code>	Checkov+tfsec → Plan → Apply → Compliance report
App CI	<code>ci-app.yml</code>	<code>src/**</code> changes	Lint → Test → Trivy → Build+Push to ECR+ACR

Pipeline	File	Trigger	What It Does
App CD	<code>cd-app.yml</code>	After CI success	Helm deploy → EKS → AKS → GKE

See [docs/RUNBOOK.md](#) for detailed operational procedures.